Practical 9: Implementation of All Scheduling Algorithms combined.

Name: Gaurav Rao

Roll No: 24293916091

Class: CSE A (IInd year)

Date: March 5, 2026

Topic: Implementation of scheduling algos combined.

1. Theory: Combined Scheduling

This practical combines all previous CPU scheduling algorithms: - **FCFS** (Practical 5) - **SJF (Preemptive & Non-Preemptive)** (Practical 6) - **Round Robin** (Practical 7) - **Priority Scheduling (Preemptive & Non-Preemptive)** (Practical 8)

A menu-driven program allows the user to select and execute any of these algorithms. This is useful for comparing their performance and behavior on the same set of processes.

Why Combine?

- To observe differences in waiting/turnaround times.
- To see how preemption and priorities affect scheduling.
- To understand practical impacts of algorithm choice.

.

2. Example Scenario (Solved)

Process	Arrival	Burst	Priority
P1	0	5	2
P2	1	3	1
P3	2	2	3
P4	3	1	2

FCFS Solution

- P1: 0-5
- P2: 5-8
- P3: 8-10
- P4: 10-11

Process	Waiting	Turnaround
P1	0	5
P2	4	7
P3	6	8
P4	7	8

SJF Solution

- P1: 0-5
- P2: 5-8
- P3: 8-10
- P4: 10-11 (Same as FCFS for this input)

Round Robin (Quantum=2)

- P1: 0-2, 8-10
- P2: 2-4, 10-11
- P3: 4-6
- P4: 6-7

Process	Waiting	Turnaround
P1	5	10
P2	6	9
P3	2	4
P4	3	4

Priority Scheduling

- P2 (priority 1): 1-4
- P1 (priority 2): 0-5
- P4 (priority 2): 5-6
- P3 (priority 3): 6-8

Process	Waiting	Turnaround
P1	0	5
P2	0	3
P3	4	6
P4	2	3

.

3. C Implementation

```
#include <stdio.h>
#define MAX 10

struct Process {
  int pid, arrival, burst, priority, waiting, turnaround, remaining,
completed;
};

void printResult(struct Process p[], int n, char* name) {
  float avg_wt = 0, avg_tat = 0;
  printf("\n%s\nPID\tAT\tBT\tWT\tTAT\n", name);
  for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\n", p[i].pid, p[i].arrival, p[i].burst,
p[i].waiting, p[i].turnaround);
    avg_wt += p[i].waiting;
    avg_tat += p[i].turnaround;
  }
  printf("\nAverage Waiting Time: %.2f", avg_wt / n);
  printf("\nAverage Turnaround Time: %.2f\n", avg_tat / n);
}

void fcfs(struct Process p[], int n) {
  int time = 0;
  printf("\nOrder of Execution:\nprocess_name\tstart\tend\n");
```

```
  for (int i = 0; i < n; i++) {
    if (time < p[i].arrival) time = p[i].arrival;
    printf("P%d\t\t%d\t%d\n", p[i].pid, time, time + p[i].burst);
    p[i].waiting = time - p[i].arrival;
    time += p[i].burst;
    p[i].turnaround = p[i].waiting + p[i].burst;
  }
  printResult(p, n, "FCFS");
}

void sjf_np(struct Process p[], int n) {
  int completed = 0, time = 0;
  for (int i = 0; i < n; i++) p[i].completed = 0;
  printf("\nOrder of Execution:\nprocess_name\tstart\tend\n");
  while (completed < n) {
    int idx = -1, min = 1e9;
    for (int i = 0; i < n; i++) {
      if (!p[i].completed && p[i].arrival <= time && p[i].burst < min) {
        min = p[i].burst; idx = i;
      }
    }
    if (idx == -1) { time++; continue; }
    printf("P%d\t\t%d\t%d\n", p[idx].pid, time, time + p[idx].burst);
    p[idx].waiting = time - p[idx].arrival;
    time += p[idx].burst;
    p[idx].turnaround = p[idx].waiting + p[idx].burst;
    p[idx].completed = 1;
    completed++;
  }
  printResult(p, n, "SJF Non-Preemptive");
}

void sjf_p(struct Process p[], int n) {
  int time = 0, completed = 0, last_idx = -1, start_time = 0;
  for (int i = 0; i < n; i++) { p[i].remaining = p[i].burst;
p[i].completed = 0; }
  printf("\nOrder of Execution:\nprocess_name\tstart\tend\n");
  while (completed < n) {
    int idx = -1, min = 1e9;
    for (int i = 0; i < n; i++) {
      if (!p[i].completed && p[i].arrival <= time && p[i].remaining <
min) {
        min = p[i].remaining; idx = i;
      }
    }
    if (idx != last_idx) {
      if (last_idx != -1) printf("P%d\t\t%d\t%d\n", p[last_idx].pid,
start_time, time);
      start_time = time;
      last_idx = idx;
    }
```

```
    if (idx == -1) { time++; continue; }
    p[idx].remaining--;
    time++;
    if (p[idx].remaining == 0) {
      p[idx].completed = 1;
      completed++;
      p[idx].turnaround = time - p[idx].arrival;
      p[idx].waiting = p[idx].turnaround - p[idx].burst;
    }
  }
  if (last_idx != -1) printf("P%d\t\t%d\t%d\n", p[last_idx].pid,
start_time, time);
  printResult(p, n, "SJF Preemptive (SRTF)");
}

void priority_np(struct Process p[], int n) {
  int completed = 0, time = 0;
  for (int i = 0; i < n; i++) p[i].completed = 0;
  printf("\nOrder of Execution:\nprocess_name\tstart\tend\n");
  while (completed < n) {
    int idx = -1, min_prio = 1e9;
    for (int i = 0; i < n; i++) {
      if (!p[i].completed && p[i].arrival <= time && p[i].priority <
min_prio) {
        min_prio = p[i].priority; idx = i;
      }
    }
    if (idx == -1) { time++; continue; }
    printf("P%d\t\t%d\t%d\n", p[idx].pid, time, time + p[idx].burst);
    p[idx].waiting = time - p[idx].arrival;
    time += p[idx].burst;
    p[idx].turnaround = p[idx].waiting + p[idx].burst;
    p[idx].completed = 1;
    completed++;
  }
  printResult(p, n, "Priority Non-Preemptive");
}

void priority_p(struct Process p[], int n) {
  int time = 0, completed = 0, last_idx = -1, start_time = 0;
  for (int i = 0; i < n; i++) { p[i].remaining = p[i].burst;
p[i].completed = 0; }
  printf("\nOrder of Execution:\nprocess_name\tstart\tend\n");
  while (completed < n) {
    int idx = -1, min_prio = 1e9;
    for (int i = 0; i < n; i++) {
      if (!p[i].completed && p[i].arrival <= time && p[i].priority <
min_prio) {
        min_prio = p[i].priority; idx = i;
      }
    }
```

```c
    if (idx != last_idx) {
      if (last_idx != -1) printf("P%d\t\t%d\t%d\n", p[last_idx].pid,
start_time, time);
      start_time = time;
      last_idx = idx;
    }
    if (idx == -1) { time++; continue; }
    p[idx].remaining--;
    time++;
    if (p[idx].remaining == 0) {
      p[idx].completed = 1;
      completed++;
      p[idx].turnaround = time - p[idx].arrival;
      p[idx].waiting = p[idx].turnaround - p[idx].burst;
    }
  }
  if (last_idx != -1) printf("P%d\t\t%d\t%d\n", p[last_idx].pid,
start_time, time);
  printResult(p, n, "Priority Preemptive");
}

void round_robin(struct Process p[], int n, int quantum) {
  int time = 0, completed = 0;
  for (int i = 0; i < n; i++) p[i].remaining = p[i].burst;
  printf("\nOrder of Execution:\nprocess_name\tstart\tend\n");
  while (completed < n) {
    int found = 0;
    for (int i = 0; i < n; i++) {
      if (p[i].remaining > 0 && p[i].arrival <= time) {
        found = 1;
        int exec = (p[i].remaining < quantum) ? p[i].remaining : quantum;
        printf("P%d\t\t%d\t%d\n", p[i].pid, time, time + exec);
        p[i].remaining -= exec;
        time += exec;
        if (p[i].remaining == 0) {
          p[i].turnaround = time - p[i].arrival;
          p[i].waiting = p[i].turnaround - p[i].burst;
          completed++;
        }
      }
    }
    if (!found) time++;
  }
  printResult(p, n, "Round Robin");
}

int main() {
  struct Process p[MAX];
  int n, choice, q;
  printf("Enter number of processes: ");
  scanf("%d", &n);
```

```
  for (int i = 0; i < n; i++) {
    p[i].pid = i + 1;
    printf("P%d Arrival, Burst, Priority: ", i + 1);
    scanf("%d %d %d", &p[i].arrival, &p[i].burst, &p[i].priority);
  }
  printf("\n1. FCFS\n2. SJF-NP\n3. SJF-P\n4. Priority-NP\n5. Priority-
P\n6. Round Robin\nChoice: ");
  scanf("%d", &choice);
  if (choice == 1) fcfs(p, n);
  else if (choice == 2) sjf_np(p, n);
  else if (choice == 3) sjf_p(p, n);
  else if (choice == 4) priority_np(p, n);
  else if (choice == 5) priority_p(p, n);
  else if (choice == 6) {
    printf("Enter Time Quantum: ");
    scanf("%d", &q);
    round_robin(p, n, q);
  }
  return 0;
}
```

.

4. Output Screenshots

- **Combined Output:**

```
~/Documents/CW/OS/practicalsAndAssignments/practicals/pr9
❯ ./a.out
Enter number of processes: 5
P1 Arrival, Burst, Priority: 2 4 2
P2 Arrival, Burst, Priority: 2 3 1
P3 Arrival, Burst, Priority: 1 6 1
P4 Arrival, Burst, Priority: 3 1 5
P5 Arrival, Burst, Priority: 2 5 4

1. FCFS
2. SJF-NP
3. SJF-P
4. Priority-NP
5. Priority-P
6. Round Robin
7. Compare All
Choice: 7
Enter Time Quantum for RR: 2

Comparison Table:
Algorithm               Avg WT          Avg TAT
FCFS                    7.60            11.40
SJF-NP                  6.40            10.20
SJF-P                   5.20            9.00
Priority-NP                     8.20            12.00
Priority-P                      7.80            11.60
Round Robin                     9.40            13.20
```